

static Polymprphism OR Compile time Polymorphism

1) The polymorphism which exist at the time of compilation is called static polymorphism.

2) Here the Java compiler will decide which method will be invoked based on the reference OR type of Parameter.

3) The binding of the method is done at the time of compilation so, it is also known as early binding.

4) Method Overloading and Method Hiding is the example of compile time polymorphism.

Dynamic Polymprphism OR Runtime Polymorphism

1) The Polymorphism which exist at the time of execution is called Dynamic polymorphism.

2) Here the Java compiler does not have any idea regarding method call, at runtime JVM will decide will method will be invoked based on the Object type.

3) The binding of the method is done at the time of execution so, it is also known as late binding.

4) Method Overriding is the example of Runtime polymorphism.

final keyword in Java :

It is used to provide some kind of restriction in our program.
We can use final keyword in ways 3 ways in java.

- 1) To declare a class as a final. (Inheritance is not possible)
- 2) To declare a method as a final (Overriding is not possible)
- 3) To declare a variable OR Field as a final (Re-assignment is not possible)

1) To declare a class as a final :

Whenever we declare a class as a final class then we can't extend or inherit that class otherwise we will get a compilation error.

Whenever the composition of the class (Logic of the class) is very important and we don't want to modify the actual behaviour of the class then we should declare a class as a final.

We can create an object for final class.

Note :- In java String and All wrapper classes are declared as final class.

```
final class A
{
    private int x = 100;

    public void setData(int x)
    {
        this.x = x;
        IO.println(this.x);
    }
}

class B extends A //error
{
}

public class FinalClassEx
{
    public static void main(String[] args)
    {
        B b1 = new B();
        b1.setData(120);
    }
}

//We can create the object for final class
final class Test
{
    private int data;

    public Test(int data)
    {
        this.data = data;
        IO.println("Data value is : "+data);
    }
}

public class FinalClassEx1
{
    public static void main(String[] args)
    {
        Test t1 = new Test(100);
    }
}
```

sealed keyword in java to work with sealed classes :

* An outer class OR top level class we can declare with public , default, abstract, final and sealed.

It is a new feature introduced from java 15v (preview version) and become the integral part of java from 17V(LTS).

It is an improvement over final keyword.

By using sealed keyword we can declare classes and interfaces as sealed.

It is one kind of restriction that describes which classes and interfaces can extend or implement from Sealed class Or interface.

It is similar to final keyword with less restriction because here we can permit the classes to extend from the original Sealed class.

The class which is inheriting from the sealed class must be final, sealed or non-sealed.

The sealed class must have atleast one sub class.

We can create object for Sealed class.

It provides the following modifiers :

- 1) sealed : Can be extended only through permitted class.
- 2) non-sealed : Can be extended by any sub class, if a user wants to give permission to its sub classes.
- 3) permits : We can provide permission to the sub classes, which are inheriting through Sealed class OR sealed interface
- 4) final : we can declare permitted sub class as final so, it cannot be extended further.

```
package com.ravi.sealed_demo;

sealed class Bird permits Parrot,Peacock
{
    public void fly()
    {
        IO.println("Generic Bird is flying");
    }
}

non-sealed class Parrot extends Bird
{
    @Override
    public void fly()
    {
        IO.println("Parrot Bird is flying");
    }
}

final class Peacock extends Bird
{
    @Override
    public void fly()
    {
        IO.println("Peacock Bird is flying");
    }
}

public class SealedExample
{
    public static void main(String[] args)
    {
        Bird b = null;
        b = new Parrot(); b.fly();
        b = new Peacock(); b.fly();
    }
}
```

2) To declare a method as a final (Overriding is not possible) :

Whenever we declare a method as a final then we can't override that method in the sub class otherwise there will be a compilation error.

We should declare a method as a final if the body of the method i.e the implementation of the method is very important and we don't want to override or change the super class method body by sub class method body then we should declare the super class method as final method.

```
class A
{
    protected int a = 10;
    protected int b = 20;

    public final void calculate()
    {
        int sum = a+b;
        IO.println("Sum is : "+sum);
    }
}

class B extends A
{
    @Override
    public void calculate() //error
    {
        int mul = a*b;
        IO.println("Mul is : "+mul);
    }
}

public class FinalMethodEx
{
    public static void main(String [] args)
    {
        A a1 = new B();
        a1.calculate();
    }
}
```

class Alpha

```
{
    private final void accept()
    {
        IO.println("Alpha class accept method");
    }
}
```

class Beta extends Alpha

```
{
    public void accept() //Re-declaration
    {
        IO.println("Beta class accept method");
    }
}
```

public class FinalMethodEx1

```
{
    public static void main(String [] args)
    {
        new Beta().accept();
    }
}
```

Note : The above code will compile and generate the output

3) To declare a variable/Field as a final :

In older languages like C and C++ we use "const" keyword to declare a constant variable but in java, const is a reserved word for future use so instead of const we should use "final" keyword.

If we declare a variable/field as a final then **we can't perform re-assignment (i.e nothing but re-initialization) of that variable.**

```
class A
{
    final int A = 10;

    public void setData()
    {
        A = 10; //error
        IO.println(A);
    }
}

class FinalVarEx
{
    public static void main(String[] args)
    {
        A a1 = new A();
        a1.setData();
    }
}

final class Student
{
    private final int rollNumber;
    private final String studentName;

    public Student(final int rollNumber, final String studentName)
    {
        this.rollNumber = rollNumber;
        this.studentName = studentName;
    }

    @Override
    public final String toString()
    {
        return "Roll is : "+this.rollNumber+"\nName is : "+this.studentName;
    }
}

class FinalVarEx1
{
    public static void main(String[] args)
    {
        final Student s1 = new Student(111, "Scott");
        IO.println(s1);

        // s1 = new Student(222, "Smith"); //Invalid
    }
}
```

Assignment : How to create an **immutable object**

Note : String and All the Wrapper classes are immutable classes

What is Method Chaining in Java :